



MULTI-LINGUAL NLP MİMARİLERİ VE DİLBİLİMSEL YAKLAŞIMLARI

MULTI-LINGUAL NLP MİMARİLERİ

PRE-TRAINED MODELS

mBERT

104 Wikipedia sayfasında bulunan farklı dillerdeki article'ları dataset olarak kullanan mBERT modeli, dataset içerisindeki tüm farklı dilleri tek bir dil olarak algılayarak çalışıyor.

Bu işlemi WordPiece adındaki tüm dillerde ortak paylaşımlı olarak yer alan bir dictionary kullanarak gerçekleştiriyor. WordPiece, içerisindeki dil verisinde yer alan cümle ve sözcükleri parçalarına ayırarak "Subword Tokenization" gerçekleştiriyor. Linguistik açıdan sözcükleri bağımlı ve bağımsız olup olmadığını ayırt etmeden biçimbirimlerine ayırarak bu biçimbirimleri corpus'taki yan yana gelme derecesine göre sıralıyor ve buradan yola çıkarak hafızasında yer almayan yeni sözcükleri ekleyebiliyor. Az bilinen sözcükleri bu biçimbirimlerin bir birleşimi olarak (Combination of Subwords) temsil edebiliyor.

Örneğin "Gözlükçü" sözcüğünü oluşturan "göz", "-lük", "-çü" biçimbirimlerinin ayrı ayrı standalone ve merged şekilde bulunma durumlarına baktıktan sonra "göz" sözcüğünün kendi başına bir sözcük olduğuna karar verip "-lük" ve "-çü" birimlerinin "göz" gibi isimler ile yan yana geldiğini fark ediyor ve bunu sözlükçesine ilgili parametreleri ile ekliyor.

Hiyerarşik olarak en uzun subword'leri önceleyerek daha kısa subword'lere doğru ilerliyor. Sonuç olarak morfolojik temelli bir tokenization metodu kullanarak algoritmasını inşa ediyor olması mBERT'i pek çok modele kıyasla ileride konumlandırabilir.

Bu tip bir algoritma metin üretimi ve benzeri görevler için yeterli görünüyor olsa da diyalog yürütme ve gerçek hayatta karşılıklı konuşma üzerinden bir görevi yerine getirmeye çalışırsa bağlam ve anlama dair bir bilgi kodlamadığı için morfemlerinden eksik bazı ifadeleri nasıl eşleştireceğini bilemeyebilir veya doğru çıktıyı üretemeyebilir.

PRE-TRAINING PROCESS

Model birden fazla dilin metnine maruz bırakılarak anlamsal olarak birbiriyle benzer olan sözcükleri vektörlere dönüştürerek kodluyor. Ardından da Subword Tokenization yöntemi ile WordPiece seti üzerinde sürekli güncellemeler ve eklemeler gerçekleştiriyor.

Vectorization işleminde birbirlerinin çevirisi olan metinleri (labeled data) kullanabiliyorken, VecMap üzerinden matrix operasyonları ile unlabeled veri ile de train edilebiliyor.

FINE-TUNING PROCESS

Pre-trained olan modele bol veriye sahip olan bir dil üzerinden (örn: İngilizce) etiketlenmiş (labeled) data kullanılarak spesifik görevler için optimize ediliyor. (Sentiment analysis, chatting vb.)

TRANSFER PROCESS

Pre-training aşamasında tüm dilleri semantik vektörler ile kodladığı için mBERT doğrudan başka bir dilde de eğitildiği anadile yakın bir performans gösterebiliyor. Bu gibi doğrudan transferler Zero-shot Transfers olarak adlandırılıyor.

Few-shots Transfer olarak adlandırılan yöntemde ise hedeflenen dile dair var olan labeled data kullanılarak modelin hedef dilin nüanslarını daha iyi kavraması sağlanıyor.

POS TAGS & DEPENDENCY STRUCTURES

POS TAGS: Dillerin evrensel özelliklerinin kodlanarak pre-training sürecinde bu yapıları modellerin tanınması sağlanıyor. Örneğin: “Ali ile dün sinemaya gittik.” Cümlesini Ali: Subject, ile: Conjunction, dün: adverb, sinemaya: object, gitti(k): verb, (-k): Secondary subject (gömülü morphem) şeklinde kodlayarak hedef dildeki benzer kodlamalara sahip sözcükleri eşleştirmesi sağlanıyor.

Dependency Structures: Cümlenin mantıksal ilişki ağına bakarak benzer mantıksal ilişkilere sahip yapıların vektörel uzayda yakın kodlanması sağlanıyor.

Örneğin: “He killed the guy” = He: Eylemi gerçekleştiren (X), killed: Gerçekleştirilen eylem (Y) the guy: Eylemden etkilenen(Z). = “O, adamı öldürdü” O: (X), adamı: (Y), öldürdü: (Z)

ADVERSARIAL LEARNING

Modeli “Generator” ve “Discriminator” olarak ikiye bölerek model train edildikten sonra yanıltılmaya çalışılıyor. Generator modelin ayırt edemeyeceği veriler oluştururken Discriminator oluşturulan bu verileri doğru sınıflandırmaya çalışır. Amaç oluşan bu fonksiyonu $\{ \min[\theta] \max[\delta \in S] L(f(x+\delta; \theta), y) \}$ minimize etmektir.

PSEUDO-LABELING

Hedef dilin unlabeled data'sına tahmini label'lar yerleştirerek modelin fine-tune'lanması sağlanır.

API AVAILABILITY AND CODE SNIPPET

mBERT, public olarak erişime açık olmakla birlikte, ilgili Python kütüphanelerinin yüklü olduğu her local ve Google Colab, AWS gibi cloud-based sistemlerde de kullanılabilir. mBERT'in kullanımı için tercih edilen popüler yöntemlerden birisinin HuggingFace Transformers library'sinin üzerinden olduğu görülüyor.

Aşağıda; Python yüklü bir sistem üzerinden modelin yüklenmesi, text tokenization işlemi, tokenized yapıların dimension reduction uygulanarak vektörel yapılara çevrilmesi ve spesifik task'lar için modelin fine-tune'lanma süreci kod üzerinden gösterilmektedir. Modelin Speech-to-Text özelliği bulunmamaktadır.

```
import torch

from transformers import BertTokenizer, BertModel

# Load the mBERT model and tokenizer

model = BertModel.from_pretrained('bert-base-multilingual-cased')

tokenizer = BertTokenizer.from_pretrained('bert-base-multilingual-cased')


# Tokenize and encode the sentence

sentence = "This is an example sentence."

input_ids = tokenizer.encode(sentence, add_special_tokens=True)

print("Input IDs:", input_ids)

# Convert input IDs to tensor and get the attention mask

input_ids = torch.tensor([input_ids])

attention_mask = torch.tensor([[1] * len(input_ids)])


# Generate embeddings

with torch.no_grad():

    outputs = model(input_ids, attention_mask=attention_mask)

    last_hidden_states = outputs.last_hidden_state

print("Last Hidden States Shape:", last_hidden_states.shape)
```

TOKENIZING AND GENERATING EMBEDDINGS FOR MULTILINGUAL TASKS

```
sentences = [

    "This is an example sentence.",    # English

    "这是一个例句。",                  # Chinese

    "Ceci est une phrase exemple.",    # French

    "Esto es una oración de ejemplo." # Spanish

]

# Tokenize and encode each sentence

for sentence in sentences:

    input_ids = tokenizer.encode(sentence, add_special_tokens=True)

    input_ids = torch.tensor([input_ids])

# Generate embeddings

with torch.no_grad():

    outputs = model(input_ids)

    last_hidden_states = outputs.last_hidden_state

print(f"Sentence: {sentence}")
print(f"Last Hidden States Shape: {last_hidden_states.shape}")
print("-" * 50)
```

FINE-TUNING THE MODEL FOR A SPECIFIC TASK (CLASSIFICATION USED AS AN EXAMPLE)

```
from transformers import BertForSequenceClassification, AdamW

# Load pre-trained mBERT model for sequence classification
model = BertForSequenceClassification.from_pretrained('bert-base-multilingual-cased', num_labels=2)

# Example data
input_ids = tokenizer.encode("This is an example.", add_special_tokens=True)
input_ids = torch.tensor([input_ids])

# Dummy label (for binary classification, e.g., 0 or 1)
labels = torch.tensor([1])

# Forward pass (this will return the loss and logits)
outputs = model(input_ids, labels=labels)
loss, logits = outputs.loss, outputs.logits
print("Loss:", loss)
print("Logits:", logits)
```

XLM-R Large

mBERT gibi XLM-R da tek bir transformer mimarisi ile ortaklaşa çalışıyor ve bu sayede bir dilden öğrenilen bilgileri farklı bir dile aktararak birden fazla dilde öğrenim gerçekleştirmesi sağlanabiliyor. Ancak mBERT'in aksine tokenization metodu olarak WordPiece değil BPE(Byte-Pair-Encoding) yöntemini kullanıyor. Bu iki yöntem de temelde cümle ve kelimeleri parçalara (biçimbirimlere/morfemlere) bölerek matematiksel bir yakınsama fonksiyonu (bknz:

$$\text{Vocabulary Update: } V_{\text{new}} = \text{Merge}(\arg \max_{s \in S} L(s))$$
) kullanıyor olsalar da, WordPiece; en sık eşleşen pair'ları belirlemek için train edildiği veriye bağlı şekilde bir "benzerlik" algoritması ile bu pair'ların eşleştirilmesine karar veriyor. BPE ise doğrudan morfemlerin yan yana gelme sıklıklarını nicel olarak inceleyip bu işlemi gerçekleştiriyor. mBERT'e kıyasla çok daha fazla data ile train edilmekle birlikte mBERT'in sahip olduğu parametrelerin 2 katı ile çalışmakta. (24 layers, 1024 hidden units) Ve bu fark XLM-R Large modelini mBERT'e göre çok daha hızlı, daha az kaynağa sahip dillerde daha doğru işleme ve genel olarak daha yüksek bir performans gerçekleştirebilmesini sağlıyor. XLM-R yine mBERT gibi pre-training sürecinde Masked Language Modeling metodunu kullanarak input token'larının %15'e yakını maskeliyerek diğer token'lar üzerinden maskelenen token'ları tahmin etmeye çalışıyor ve bu sayede contextual verisini arttırarak context'e dayalı iletişimlerde daha iyi performans göstermesi amaçlanıyor. mBERT'de olduğu gibi XLM-R Large'da da Speech-to-Text özelliği bulunmamaktadır.

API AVAILABILITY AND CODE SNIPPET

XLM-R LARGE public bir model olmakla birlikte local'e ve Cloud servislere deploy edilebilmekte. HuggingFace API'si üzerinden veya custom şekilde modeli import ederek istenilen ortamda çalıştırılabilir.

```
from transformers import XLMRobertaTokenizer, XLMRobertaForSequenceClassification
import torch

# Load the tokenizer and model
tokenizer = XLMRobertaTokenizer.from_pretrained('xlm-roberta-base')
model = XLMRobertaForSequenceClassification.from_pretrained('xlm-roberta-base')

# Encode input text
inputs = tokenizer("This is a test sentence.", return_tensors="pt")

# Get predictions
outputs = model(**inputs)
logits = outputs.logits
```

ByT5

Diğer modellerin aksine ByT5, metni morfe bazlı tokenize etmek yerine doğrudan byte seviyesinde işliyor. Yapıyı parçalara bölerek morfemleri temsil etmek yerine o yapıyı temsil eden byte sequence'ını kullanıyor. Bu durum karmaşık morfe yapısına sahip dillerde ortaya çıkan hatalı tokenization problemini çözebiliyor. Byte seviyesinde dil işleme gücü sayesinde ByT5'in bir diğer avantajı da bu sayede multi-lingual dillerde de iyi performans gösterebiliyor olması. Temel algoritmasının belirli bir dil üzerine inşa edilmemesi onu evrensel bir NLP modeli olma konumuna taşıyor. Öncülü T5 gibi ByT5 de Encoder ve Decoder mimarisi ile algoritmasını çalıştırıyor.

Çeviri, sınıflandırma, soru-cevap ve özetleme gibi işlemleri herhangi bir dil için pre-processing'e ihtiyaç duymadan yapabiliyor. Fakat Speech-to-Text özelliği barındırmaması ve byte level işleme süreci tokenization işleme sürecine kıyasla daha maliyetli olduğu için tüm bu avantajlarına rağmen çalışılan projenin yapısına, maliyetine ve kazanç durumuna göre kullanımı dezavantajlı duruma düşebilir.

API AVAILABILITY & CODE SNIPPET

Modele doğrudan public olarak erişebilmek mümkün olmasa da HuggingFace transformer'ları aracılığı ile Python environment'ına yüklenerek istenilen platformlarda local ya da Cloud olarak kullanılabilmesi mümkün.

```
from transformers import ByT5Tokenizer, ByT5ForConditionalGeneration

# Load the ByT5 model and tokenizer

tokenizer = ByT5Tokenizer.from_pretrained("google/byt5-small")
model = ByT5ForConditionalGeneration.from_pretrained("google/byt5-small")

# Example input text
input_text = "Translate English to French: How are you?"

# Tokenize the input
inputs = tokenizer(input_text, return_tensors="pt")

# Generate output
outputs = model.generate(**inputs)

# Decode the generated output
decoded_output = tokenizer.decode(outputs[0], skip_special_tokens=True)

print("Input Text: ", input_text)
print("Generated Output: ", decoded_output)
```

GPT

GPT de tokenization işlemi için BPE metodunun bir varyantını kullanıyor. En sık yan yana gelen morfemleri doğrudan sıklık derecesine bakarak eşleştiriyor. Onu aynı tokenization yöntemini kullanan modellerden çok daha önde tutan olgu ise GPT'nin sahip olduğu mimari. Tokenization işlemi sırasında multi weight hesaplaması yaparak her kelimeyi Query (Q), Key(K) ve Value(V) şeklinde üç vektöre

$$\text{Attention Score}_{ij} = \frac{Q_i K_j^T}{\sqrt{d_k}}$$

ayırır. Her W_x ile W_y için Attention Score'u hesaplar. Attention Score Ardından çıkan skor softmax fonksiyonu ile ihtimal hesabı üzerinden 1 ve 0 arasına sıkıştırılır. Tüm bu süreç $W * W(n1) * W(n2) \dots * W(nn)$ serisine uygulanır. Sıralı morfemlerin kendi arasındaki yakınlık ilişkisini çok daha optimize bir şekilde hesapladıktan sonra Attention Score dağılımlı olarak hesaplandığı için W'lerin pozisyonel bilgisine Self-Attention mekanizmaları doğrudan erişemez. Bunun için Positional Encoding kullanılarak hesaplanan W'lerin pozisyon bilgileri ilgili W'lere dağıtılır. Self-Attention Score hesaplamasında GPT; hesaplamayı RNN mimarileri gibi sıralı şekilde değil, eş zamanlı ve paralel yapıda gerçekleştirir, bu da hız ve optimizasyondan kazandırırken, bazı parametrik bilgilerin beraberinde hesaplanamamasına sebep olur. Bu paketlenmiş bilgiler ilgili neural networkler'e iletilir ve weight/bias'lar dağıtılır.

API AVAILABILITY & CODE SNIPPET

ChatGPT, OpenAI üzerinden ilgili fiyat paketleri ile API key edinilerek istenilen ortamda kullanılabilen public bir yapıya sahiptir. Örnek kod:

```
import openai

# Replace with your own API key
openai.api_key = 'your-api-key-here'

response = openai.ChatCompletion.create(
    model="gpt-4",
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Hello! How can I use the ChatGPT API?"},
    ]
)

print(response['choices'][0]['message']['content'])
```

MEVCUT NLP MODELLERİNİN MİMARİLERİNE DAİR DEĞERLENDİRME

Çoğu modelin mimarisi dil verisini işlemek için biçimbilimsel bir yaklaşım sergileyerek onları tokenize ediyor. Bir dilin biçimsel yapısı o dil hakkında birçok yapısal ve semantik bilgiyi kodlayabilse de anlam ve bağlam gibi bilgilerin kodlanabilmesinde doğrudan yardımcı olamaz. Çünkü dil nedensizdir. “Masa” sözcüğünün neden gerçek dünyadaki masayı temsil ettiğine dair bir bulgu yoktur, bulunamaz. Zihin soyut bir şekilde gerçek dünyadaki izdüşümlerin önce kavramsal bilgisini, ardından da onu temsil eden sesleri zihninde eşleştirerek kodlar. Fakat bu üretim ve eşleştirme aşamaları tamamen nedensiz ve bağıntısızdır. (Anlamlı bir korelasyon içermezler). BPE gibi doğrudan korelasyon odaklı morfeim sondalarının bu kadar verimli çalışabilmesinin bir nedeni de sözcükler ve anlamları arasında anlamsal değil, kullanımsal bir korelasyon olmasıdır.

Sözcük bazında ilişkileri tespit etme konusunda her ne kadar mevcut modeller gayet güçlü olsa da temelde yapılan işlem veriye yapılan istatistiksel bir analiz olduğu için mevcut epistemolojik yöntem ile (Bknz: istatistiksel fonksiyonların optimizasyonu ve birden fazla lineer denklemin toplamları ile inşa edilen neural network’ler) gerçekten çıkarım yapabilen, “anladığı” için cevap veren bir yapay “zihin” elde edebilmek mümkün görünmüyor. Bunun yerine insan zihni gibi konuşabilen bir sistemin ötesine geçilememesi pek olası. (Bknz: [Chinese Room Experiment](#)).

BİRDEN ÇOK DİL EDİNİM VE ÖĞRENİMİ AÇISINDAN İNSAN VE MAKİNE

Yeni doğan bir bebek ilk dilini sadece çevresinden gelen dilsel girdiye maruz kalarak öğrenir. Bu tür çevresel bir girdinin eksikliğinde ise kişi son derece eksik bilişsel fonksiyonlara sahip olur. Yeni doğan bebek ilk edindiği dilini doğrudan öğrenir, yani parametrelerini doğrudan ilgili dilden oluşturur ve o dilde düşünür. Çok kısa bir sürede ilgili dilde çok hızlı bir üretkenlik yetisi kazanır. Fakat ilk dil edinildikten sonra eğer kişi 1-2 yaşını geçtiyse, Nöroplastisite önemli ölçüde düştüğü için artık dilleri edinmez, öğrenir.

Öğrenim ile edinim arasındaki en temel fark ise ilke ve parametre farkıdır. Her insan doğuştan insan dilini (hangi dil olduğu fark etmeksizin) inanılmaz hızlı ve optimize bir şekilde edinebilme yetisine sahiptir. İlk dil edinilirken zihinde doğuştan var olan bir dil edinim algoritması (Language Acquisition Device) kullanılır. Bu algoritma ilkeler denilen yapıları içerir. İlkeler; mantıksal ilişkiler kurabilme, özne, nesne, yüklem yapılarını ayırt edebilme, evrensel dilsel bilgi yapısında üretim yapabilme gibi yetileri içerir. Bu açıdan beynin dilleri tokenize ederek değil, doğrudan salt byte temsillerini işlemediği söylenebilir. (Bknz: ByT5) Evrensel dilsel bilgi yapısı, tüm cümlelerin 3'ten fazla üyesi olamayacağını savunur. Örneğin: "Aslı yaşlandı." Cümlesi tek üyeli bir yapıdır. X: Aslı = Yaşlanan. "Aslı köpeği seviyor." Cümlesi ise 2 üyeli bir yapıdır. X: Aslı = Seven, Y: köpek = Sevilen. Son olarak "Aslı kitabı Ali'ye verdi." Cümlesi 3 üyeli bir yapıdır. X: Aslı = [Kitabı] Veren, Y: Kitap = Verilen, Z: Ali = [Kitabı] Alan. Bugüne dek dünyadaki hiçbir dilin 3'ten fazla bilgi taşıyabilen bir cümle yapısına sahip olduğu görülmemiştir. Tüm dilsel enformasyon en fazla 3, en az 1 temel bilgi üyesine indirgenebilir yapıdadır. İnsan zihni, evrenin temel bilgi iletim şeması nedeniyle dillerini bu şekilde kodlamıştır. Fakat makineler böyle bir enformatik perspektif içermediği için bu bilgi mimarilerine entegre edilememiştir.

İnsan zihni ikinci bir dil öğrenmeye başladığında ise öncelikle anadilinden edinmiş olduğu parametreler aracılığı ile ikinci dili öğrenmeye başlar. Örneğin Türkçe anadili ile Fransızca öğrenmeye başlayan bir kişi, Türkçede kişi ve tekil/çoğul artikelleri sadece tek bir kişi zamiri dışında bulunmadığı için (He, she, it = O, le, la, les = None) kişi başta sözcükleri artikelleri ile öğrenmekte zorluk yaşayacaktır. Bir diğer örnek ise Türkçede sıfatlar isimlerden önce geldiği için Fransızcadaki isim + sıfat kodlamasına alışması zaman alacak ve bu yüzden Fransızca bir cümleyi telaffuz ederken syntax olarak Türkçeden parametre transformation'ı gerçekleştirecektir. Örn: Elle lit un livre rouge [Doğru parametre dönüşümü] = O okuyor bir kitap kırmızı [Hatalı parametre dönüşümü]. Cümlesini "Elle rouge un livre lit." [Hatalı parametre dönüşümü] olarak sesletecektir.

Bu açıdan NLP mimarilerinde kullanılan Few-shots learning metodu ile insan zihni benzerlik gösterir. Yeni bir dil öğrenirken mevcut dilden gelen bilgi ile hedef dilden sahip olduğu minimal bilgiyi birleştirerek üretim gerçekleştirir. İnsan zihni nöroplastisitesi yüksekken unsupervised learning ile zihnini train ederken, nöroplastisite düştükçe motivasyon ve toplumsal nedenlerle de birlikte (Okullardaki zorunlu yabancı dil eğitimi, bireysel sebeplerce öğrenilen diller için alınan dersler vb.) supervised learning metodu ile de bir dil edinebilir fakat bu bir training sürecinden ziyade artık bir fine-tuning sürecidir. Çünkü gerekli parametre ve weight/bias değerleri zaten öncesinde oluşturulmuştur. Bu açıdan bazı multi-lingual NLP modellerinin; supervised based fine-tuning ile bir dilin öğrenimini gerçekleştirmesi mimari açıdan insan zihni ile benzerlik gösterebilmektedir.

SONUÇ

İnsan zihni; dil edinim süresince NLP modelleri gibi uyaran/girdi sıklığını bir kriter olarak kullanıyor olsa da, zamanla öğrenim algoritmasını sıklıktan kaydırarak kendine özgün olan parametreler ile değiştiriyor. Bu parametreleri ise anlam, kültür ve bağlam gibi toplumsal ve bireysel izdüşümler oluşturuyor. Dolayısıyla NLP modellerinin aksine saf istatistiksel denklemlerin üzerine çok daha kompleks ve kaotik bilgilerin hesaplamalarını inşa ederek üretici ve sürekli güncel kalan bir sistem oluşturuyor.

Şu anki modellerin çok daha ötesinde bir model geliştirilmek isteniyorsa, mevcut mimarilerin sadece daha kompleks hale getirilmesi nihai bir çözüm olmayacaktır. Bu süreç maliyeti arttırmakla birlikte sadece taklidi daha “iyi” hale getirecektir.

SÖZLÜKÇE

Morfoloji: Biçimbilim. Dil veya başka bir sistemin yapısal incelendiği bilim dalı. Dilbilimin 6 temel alanından biri.

Morfem/Biçimbirim: Anlamlı en küçük yapı. Bknz: Sular kesildi = Su(isim)-lar(çoğul) kes(fil)-il(edilgen)-di(geçmiş zaman)

Sonda: İnsan zihninde belirli bir syntax yapısı ile eşleşmeye programlanmış dilsel parçalar.

Language Acquisition Device: İnsan zihninin doğuştan sahip olduğu dil edinim algoritması.